JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class ConcurrentSkipListSet<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractSet<E>
      java.util.concurrent.ConcurrentSkipListSet<E>

Type Parameters:

`E - the type of elements maintained by this set`

All Implemented Interfaces:

`Serializable, Cloneable, Iterable<E>, Collection<E>, NavigableSet<E>, Set<E>, SortedSet<E>`

```
public class ConcurrentSkipListSet<E>
extends AbstractSet<E>
implements NavigableSet<E>, Cloneable, Serializable
```

A scalable concurrent `NavigableSet` implementation based on a `ConcurrentSkipListMap`. The elements of the set are kept sorted according to their natural ordering, or by a `Comparator` provided at set creation time, depending on which constructor is used.

This implementation provides expected average *log(n)* time cost for the `contains`, `add`, and `remove` operations and their variants. Insertion, removal, and access operations safely execute concurrently by multiple threads.

Iterators and spliterators are *weakly consistent*.

Ascending ordered views and their iterators are faster than descending ones.

Beware that, unlike in most collections, the `size` method is *not* a constant-time operation. Because of the asynchronous nature of these sets, determining the current number of elements requires a traversal of the elements, and so may report inaccurate results if this collection is modified during traversal.

Additionally, the bulk operations `addAll`, `removeAll`, `retainAll`, `containsAll`, `equals`, and `toArray` are *not* guaranteed to be performed atomically. For example, an iterator operating concurrently with an `addAll` operation might view only some of the added elements.

This class and its iterators implement all of the *optional* methods of the `Set` and `Iterator` interfaces. Like most other concurrent collection implementations, this class does not permit the use of `null` elements, because `null` arguments and return values cannot be reliably distinguished from the absence of elements.

This class is a member of the Java Collections Framework.

Since:

`1.6`

See Also:

`Serialized Form`

Constructor Summary

Constructors

Constructor	Description
`ConcurrentSkipListSet()`	Constructs a new, empty set that orders its elements according to their **natural ordering**.

`ConcurrentSkipListSet(Collection<? extends E> c)`	Constructs a new set containing the elements in the specified collection, that orders its elements according to their **natural ordering**.
`ConcurrentSkipListSet(Comparator<? super E> comparator)`	Constructs a new, empty set that orders its elements according to the specified comparator.
`ConcurrentSkipListSet(SortedSet<E> s)`	Constructs a new set containing the same elements and using the same ordering as the specified sorted set.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`boolean`	`add(E e)`	

		Adds the specified element to this set if it is not already present.
E	**ceiling**(**E** e)	Returns the least element in this set greater than or equal to the given element, or `null` if there is no such element.
void	**clear**()	Removes all of the elements from this set.
ConcurrentSkipListSet<**E**>	**clone**()	Returns a shallow copy of this `ConcurrentSkipListSet` instance.
Comparator<? super **E**>	**comparator**()	Returns the comparator used to order the elements in this set, or `null` if this set uses the **natural ordering** of its elements.
boolean	**contains**(**Object** o)	Returns `true` if this set contains the specified element.
Iterator<**E**>	**descendingIterator**()	Returns an iterator over the elements in this set in descending order.
NavigableSet<**E**>	**descendingSet**()	Returns a reverse order view of the elements contained in this set.
boolean	**equals**(**Object** o)	Compares the specified object with this set for equality.
E	**first**()	Returns the first (lowest) element currently in this set.
E	**floor**(**E** e)	Returns the greatest element in this set less than or equal to the given element, or `null` if

		there is no such element.
NavigableSet<E>	**headSet(E** toElement)	Returns a view of the portion of this set whose elements are strictly less than `toElement`.
NavigableSet<E>	**headSet(E** toElement, boolean inclusive)	Returns a view of the portion of this set whose elements are less than (or equal to, if `inclusive` is true) `toElement`.
E	**higher(E** e)	Returns the least element in this set strictly greater than the given element, or `null` if there is no such element.
boolean	**isEmpty**()	Returns `true` if this set contains no elements.
Iterator<E>	**iterator**()	Returns an iterator over the elements in this set in ascending order.
E	**last**()	Returns the last (highest) element currently in this set.
E	**lower(E** e)	Returns the greatest element in this set strictly less than the given element, or `null` if there is no such element.
E	**pollFirst**()	Retrieves and removes the first (lowest) element, or returns `null` if this set is empty.
E	**pollLast**()	Retrieves and removes the last (highest) element, or returns `null` if this set is empty.

boolean	**remove**(**Object** o)	Removes the specified element from this set if it is present.
boolean	**removeAll**(**Collection**<?> c)	Removes from this set all of its elements that are contained in the specified collection.
int	**size**()	Returns the number of elements in this set.
Spliterator<**E**>	**spliterator**()	Returns a **Spliterator** over the elements in this set.
NavigableSet<**E**>	**subSet**(**E** fromElement, boolean fromInclusive, **E** toElement, boolean toInclusive)	Returns a view of the portion of this set whose elements range from `fromElement` to `toElement`.
NavigableSet<**E**>	**subSet**(**E** fromElement, **E** toElement)	Returns a view of the portion of this set whose elements range from `fromElement`, inclusive, to `toElement`, exclusive.
NavigableSet<**E**>	**tailSet**(**E** fromElement)	Returns a view of the portion of this set whose elements are greater than or equal to `fromElement`.
NavigableSet<**E**>	**tailSet**(**E** fromElement, boolean inclusive)	Returns a view of the portion of this set whose elements are greater than (or equal to, if `inclusive` is true) `fromElement`.

Methods inherited from class java.util.AbstractSet

`hashCode`

Methods inherited from class java.util.AbstractCollection

`addAll`, `containsAll`, `retainAll`, `toArray`, `toArray`, `toString`

Methods inherited from class java.lang.Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.Set

addAll, containsAll, hashCode, retainAll, toArray, toArray

Methods inherited from interface java.util.Collection

parallelStream, removeIf, stream

Methods inherited from interface java.lang.Iterable

forEach

Constructor Detail

ConcurrentSkipListSet

```
public ConcurrentSkipListSet()
```

Constructs a new, empty set that orders its elements according to their natural ordering.

ConcurrentSkipListSet

```
public ConcurrentSkipListSet(Comparator<? super E> comparator)
```

Constructs a new, empty set that orders its elements according to the specified comparator.

Parameters:

`comparator - the comparator that will be used to order this set. If null, the natural ordering of the elements will be used.`

ConcurrentSkipListSet

```
public ConcurrentSkipListSet(Collection<? extends E> c)
```

Constructs a new set containing the elements in the specified collection, that orders its elements according to their natural ordering.

Parameters:

c - The elements that will comprise the new set

Throws:

ClassCastException - if the elements in c are not Comparable, or are not mutually comparable

NullPointerException - if the specified collection or any of its elements are null

ConcurrentSkipListSet

public ConcurrentSkipListSet(SortedSet<E> s)

Constructs a new set containing the same elements and using the same ordering as the specified sorted set.

Parameters:

s - sorted set whose elements will comprise the new set

Throws:

NullPointerException - if the specified sorted set or any of its elements are null

Method Detail

clone

public ConcurrentSkipListSet<E> clone()

Returns a shallow copy of this ConcurrentSkipListSet instance. (The elements themselves are not cloned.)

Overrides:

clone in class Object

Returns:

a shallow copy of this set

See Also:

Cloneable

size

public int size()

Returns the number of elements in this set. If this set contains more than `Integer.MAX_VALUE` elements, it returns `Integer.MAX_VALUE`.

Beware that, unlike in most collections, this method is *NOT* a constant-time operation. Because of the asynchronous nature of these sets, determining the current number of elements requires traversing them all to count them. Additionally, it is possible for the size to change during execution of this method, in which case the returned result will be inaccurate. Thus, this method is typically not very useful in concurrent applications.

Specified by:

`size` in interface `Collection<E>`

Specified by:

`size` in interface `Set<E>`

Specified by:

`size` in class `AbstractCollection<E>`

Returns:

the number of elements in this set

isEmpty

```
public boolean isEmpty()
```

Returns `true` if this set contains no elements.

Specified by:

`isEmpty` in interface `Collection<E>`

Specified by:

`isEmpty` in interface `Set<E>`

Overrides:

`isEmpty` in class `AbstractCollection<E>`

Returns:

true if this set contains no elements

contains

```
public boolean contains(Object o)
```

Returns `true` if this set contains the specified element. More formally, returns `true` if and only if this set contains an element `e` such that `o.equals(e)`.

Specified by:

`contains` in interface `Collection<E>`

Specified by:

`contains` in interface `Set<E>`

Overrides:

`contains` in class `AbstractCollection<E>`

Parameters:

`o` - object to be checked for containment in this set

Returns:

`true` if this set contains the specified element

Throws:

`ClassCastException` - if the specified element cannot be compared with the elements currently in this set

`NullPointerException` - if the specified element is null

add

```
public boolean add(E e)
```

Adds the specified element to this set if it is not already present. More formally, adds the specified element `e` to this set if the set contains no element `e2` such that `e.equals(e2)`. If this set already contains the element, the call leaves the set unchanged and returns `false`.

Specified by:

`add` in interface `Collection<E>`

Specified by:

`add` in interface `Set<E>`

Overrides:

`add` in class `AbstractCollection<E>`

Parameters:

`e` - element to be added to this set

Returns:

`true` if this set did not already contain the specified element

Throws:

`ClassCastException` - if `e` cannot be compared with the elements currently in this set

`NullPointerException` - if the specified element is null

remove

```
public boolean remove(Object o)
```

Removes the specified element from this set if it is present. More formally, removes an element `e` such that `o.equals(e)`, if this set contains such an element. Returns `true` if this set contained the element (or equivalently, if this set changed as a result of the call). (This set will not contain the element once the call returns.)

Specified by:

`remove` in interface `Collection<E>`

Specified by:

`remove` in interface `Set<E>`

Overrides:

`remove` in class `AbstractCollection<E>`

Parameters:

`o - object to be removed from this set, if present`

Returns:

`true if this set contained the specified element`

Throws:

`ClassCastException - if o cannot be compared with the elements currently in this set`

`NullPointerException - if the specified element is null`

clear

```
public void clear()
```

Removes all of the elements from this set.

Specified by:

`clear` in interface `Collection<E>`

Specified by:

`clear` in interface `Set<E>`

Overrides:

`clear` in class `AbstractCollection<E>`

iterator

```
public Iterator<E> iterator()
```

Returns an iterator over the elements in this set in ascending order.

Specified by:

`iterator` in interface `Iterable<E>`

Specified by:

`iterator` in interface `Collection<E>`

Specified by:

`iterator` in interface `NavigableSet<E>`

Specified by:

`iterator` in interface `Set<E>`

Specified by:

`iterator` in class `AbstractCollection<E>`

Returns:

an iterator over the elements in this set in ascending order

descendingIterator

```
public Iterator<E> descendingIterator()
```

Returns an iterator over the elements in this set in descending order.

Specified by:

`descendingIterator` in interface `NavigableSet<E>`

Returns:

an iterator over the elements in this set in descending order

equals

```
public boolean equals(Object o)
```

Compares the specified object with this set for equality. Returns `true` if the specified object is also a set, the two sets have the same size, and every member of the specified set is contained in this set (or equivalently, every member of this set is contained in the specified set). This definition ensures that the equals method works properly across different implementations of the set interface.

Specified by:

`equals` in interface `Collection<E>`

Specified by:

`equals` in interface `Set<E>`

Overrides:

`equals` in class `AbstractSet<E>`

Parameters:

o - the object to be compared for equality with this set

Returns:

true if the specified object is equal to this set

See Also:

Object.hashCode(), HashMap

removeAll

public boolean removeAll(Collection<?> c)

Removes from this set all of its elements that are contained in the specified collection. If the specified collection is also a set, this operation effectively modifies this set so that its value is the *asymmetric set difference* of the two sets.

Specified by:

removeAll in interface Collection<E>

Specified by:

removeAll in interface Set<E>

Overrides:

removeAll in class AbstractSet<E>

Parameters:

c - collection containing elements to be removed from this set

Returns:

true if this set changed as a result of the call

Throws:

ClassCastException - if the types of one or more elements in this set are incompatible with the specified collection

NullPointerException - if the specified collection or any of its elements are null

See Also:

AbstractCollection.remove(Object), AbstractCollection.contains(Object)

lower

public E lower(E e)

Description copied from interface: NavigableSet

Returns the greatest element in this set strictly less than the given element, or `null` if there is no such element.

Specified by:

`lower` in interface `NavigableSet<E>`

Parameters:

`e` - the value to match

Returns:

the greatest element less than e, or null if there is no such element

Throws:

`ClassCastException` - if the specified element cannot be compared with the elements currently in the set

`NullPointerException` - if the specified element is null

floor

```
public E floor(E e)
```

Description copied from interface: `NavigableSet`

Returns the greatest element in this set less than or equal to the given element, or `null` if there is no such element.

Specified by:

`floor` in interface `NavigableSet<E>`

Parameters:

`e` - the value to match

Returns:

the greatest element less than or equal to e, or null if there is no such element

Throws:

`ClassCastException` - if the specified element cannot be compared with the elements currently in the set

`NullPointerException` - if the specified element is null

ceiling

```
public E ceiling(E e)
```

Description copied from interface: `NavigableSet`

Returns the least element in this set greater than or equal to the given element, or `null` if there is no such element.

Specified by:

`ceiling` in interface `NavigableSet<E>`

Parameters:

`e` - the value to match

Returns:

the least element greater than or equal to e, or null if there is no such element

Throws:

`ClassCastException` - if the specified element cannot be compared with the elements currently in the set

`NullPointerException` - if the specified element is null

higher

```
public E higher(E e)
```

Description copied from interface: `NavigableSet`

Returns the least element in this set strictly greater than the given element, or `null` if there is no such element.

Specified by:

`higher` in interface `NavigableSet<E>`

Parameters:

`e` - the value to match

Returns:

the least element greater than e, or null if there is no such element

Throws:

`ClassCastException` - if the specified element cannot be compared with the elements currently in the set

`NullPointerException` - if the specified element is null

pollFirst

```
public E pollFirst()
```

Description copied from interface: `NavigableSet`

Retrieves and removes the first (lowest) element, or returns `null` if this set is empty.

Specified by:

`pollFirst` in interface `NavigableSet<E>`

Returns:

the first element, or null if this set is empty

pollLast

```
public E pollLast()
```

Description copied from interface: NavigableSet

Retrieves and removes the last (highest) element, or returns `null` if this set is empty.

Specified by:

`pollLast` in interface `NavigableSet<E>`

Returns:

the last element, or null if this set is empty

comparator

```
public Comparator<? super E> comparator()
```

Description copied from interface: SortedSet

Returns the comparator used to order the elements in this set, or `null` if this set uses the natural ordering of its elements.

Specified by:

`comparator` in interface `SortedSet<E>`

Returns:

the comparator used to order the elements in this set, or null if this set uses the natural ordering of its elements

first

```
public E first()
```

Description copied from interface: SortedSet

Returns the first (lowest) element currently in this set.

Specified by:

`first` in interface `SortedSet<E>`

Returns:

the first (lowest) element currently in this set

Throws:

`NoSuchElementException` - if this set is empty

last

```
public E last()
```

Description copied from interface: SortedSet

Returns the last (highest) element currently in this set.

Specified by:

`last` in interface `SortedSet<E>`

Returns:

the last (highest) element currently in this set

Throws:

`NoSuchElementException` - if this set is empty

subSet

```
public NavigableSet<E> subSet(E fromElement,
                              boolean fromInclusive,
                              E toElement,
                              boolean toInclusive)
```

Description copied from interface: NavigableSet

Returns a view of the portion of this set whose elements range from `fromElement` to `toElement`. If `fromElement` and `toElement` are equal, the returned set is empty unless `fromInclusive` and `toInclusive` are both true. The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Specified by:

`subSet` in interface `NavigableSet<E>`

Parameters:

`fromElement` - low endpoint of the returned set

`fromInclusive` - true if the low endpoint is to be included in the returned view

`toElement` - high endpoint of the returned set

`toInclusive` - true if the high endpoint is to be included in the returned view

Returns:

a view of the portion of this set whose elements range from fromElement, inclusive, to toElement, exclusive

Throws:

ClassCastException - if fromElement and toElement cannot be compared to one another using this set's comparator (or, if the set has no comparator, using natural ordering). Implementations may, but are not required to, throw this exception if fromElement or toElement cannot be compared to elements currently in the set.

NullPointerException - if fromElement or toElement is null

IllegalArgumentException - if fromElement is greater than toElement; or if this set itself has a restricted range, and fromElement or toElement lies outside the bounds of the range.

headSet

```
public NavigableSet<E> headSet(E toElement,
                               boolean inclusive)
```

Description copied from interface: NavigableSet

Returns a view of the portion of this set whose elements are less than (or equal to, if `inclusive` is true) `toElement`. The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Specified by:

headSet in interface NavigableSet<E>

Parameters:

toElement - high endpoint of the returned set

inclusive - true if the high endpoint is to be included in the returned view

Returns:

a view of the portion of this set whose elements are less than (or equal to, if inclusive is true) toElement

Throws:

ClassCastException - if toElement is not compatible with this set's comparator (or, if the set has no comparator, if toElement does not implement Comparable). Implementations may, but are not required to, throw this exception if toElement cannot be compared to elements currently in the set.

NullPointerException - if toElement is null

IllegalArgumentException - if this set itself has a restricted range, and toElement lies outside the bounds of the range

tailSet

```
public NavigableSet<E> tailSet(E fromElement,
                               boolean inclusive)
```

Description copied from interface: NavigableSet

Returns a view of the portion of this set whose elements are greater than (or equal to, if `inclusive` is true) `fromElement`. The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Specified by:

`tailSet` in interface `NavigableSet<E>`

Parameters:

`fromElement` - low endpoint of the returned set

`inclusive` - true if the low endpoint is to be included in the returned view

Returns:

a view of the portion of this set whose elements are greater than or equal to fromElement

Throws:

`ClassCastException` - if fromElement is not compatible with this set's comparator (or, if the set has no comparator, if fromElement does not implement `Comparable`). Implementations may, but are not required to, throw this exception if fromElement cannot be compared to elements currently in the set.

`NullPointerException` - if fromElement is null

`IllegalArgumentException` - if this set itself has a restricted range, and fromElement lies outside the bounds of the range

subSet

```
public NavigableSet<E> subSet(E fromElement,
                              E toElement)
```

Description copied from interface: NavigableSet

Returns a view of the portion of this set whose elements range from `fromElement`, inclusive, to `toElement`, exclusive. (If `fromElement` and `toElement` are equal, the returned set is empty.) The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-

versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Equivalent to `subSet(fromElement, true, toElement, false)`.

Specified by:

`subSet` in interface `NavigableSet<E>`

Specified by:

`subSet` in interface `SortedSet<E>`

Parameters:

`fromElement - low endpoint (inclusive) of the returned set`

`toElement - high endpoint (exclusive) of the returned set`

Returns:

`a view of the portion of this set whose elements range from fromElement, inclusive, to toElement, exclusive`

Throws:

`ClassCastException - if fromElement and toElement cannot be compared to one another using this set's comparator (or, if the set has no comparator, using natural ordering). Implementations may, but are not required to, throw this exception if fromElement or toElement cannot be compared to elements currently in the set.`

`NullPointerException - if fromElement or toElement is null`

`IllegalArgumentException - if fromElement is greater than toElement; or if this set itself has a restricted range, and fromElement or toElement lies outside the bounds of the range`

headSet

`public NavigableSet<E> headSet(E toElement)`

Description copied from interface: `NavigableSet`

Returns
a view of the portion of this set whose elements are strictly less than `toElement`. The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Equivalent to `headSet(toElement, false)`.

Specified by:

`headSet` in interface `NavigableSet<E>`

Specified by:

`headSet` in interface `SortedSet<E>`

Parameters:

`toElement` - high endpoint (exclusive) of the returned set

Returns:

a view of the portion of this set whose elements are strictly less than toElement

Throws:

`ClassCastException` - if toElement is not compatible with this set's comparator (or, if the set has no comparator, if toElement does not implement `Comparable`). Implementations may, but are not required to, throw this exception if toElement cannot be compared to elements currently in the set.

`NullPointerException` - if toElement is null

`IllegalArgumentException` - if this set itself has a restricted range, and toElement lies outside the bounds of the range

tailSet

```
public NavigableSet<E> tailSet(E fromElement)
```

Description copied from interface: `NavigableSet`

Returns a view of the portion of this set whose elements are greater than or equal to `fromElement`. The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Equivalent to `tailSet(fromElement, true)`.

Specified by:

`tailSet` in interface `NavigableSet<E>`

Specified by:

`tailSet` in interface `SortedSet<E>`

Parameters:

`fromElement` - low endpoint (inclusive) of the returned set

Returns:

a view of the portion of this set whose elements are greater than or equal to fromElement

Throws:

`ClassCastException` - if fromElement is not compatible with this set's comparator (or, if the set has no comparator, if fromElement does not implement `Comparable`). Implementations may, but are not required to, throw this exception if fromElement cannot be compared to elements currently in the set.

`NullPointerException` - if fromElement is null

`IllegalArgumentException` - if this set itself has a restricted range, and fromElement lies outside the bounds of the range

descendingSet

```
public NavigableSet<E> descendingSet()
```

Returns a reverse order view of the elements contained in this set. The descending set is backed by this set, so changes to the set are reflected in the descending set, and vice-versa.

The returned set has an ordering equivalent to `Collections.reverseOrder(comparator())`. The expression `s.descendingSet().descendingSet()` returns a view of `s` essentially equivalent to `s`.

Specified by:

`descendingSet` in interface `NavigableSet<E>`

Returns:

a reverse order view of this set

spliterator

```
public Spliterator<E> spliterator()
```

Returns a `Spliterator` over the elements in this set.

The `Spliterator` reports `Spliterator.CONCURRENT`, `Spliterator.NONNULL`, `Spliterator.DISTINCT`, `Spliterator.SORTED` and `Spliterator.ORDERED`, with an encounter order that is ascending order. Overriding implementations should document the reporting of additional characteristic values.

The spliterator's comparator (see `Spliterator.getComparator()`) is `null` if the set's comparator (see `comparator()`) is `null`. Otherwise, the spliterator's comparator is the same as or imposes the same total ordering as the set's comparator.

Specified by:

`spliterator` in interface `Iterable<E>`

Specified by:

`spliterator` in interface `Collection<E>`

Specified by:

`spliterator` in interface `Set<E>`

Specified by:

`spliterator` in interface `SortedSet<E>`

Returns:

a Spliterator over the elements in this set

Since:

1.8

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.